*

VLC

https://github.com/jurdabos/vlc

For the course DLBDSEDE02 – tutor: Dr. Sahar Qaadan

STUDENT: TORDA BALÁZS, 92128244 (Data Science Bsc.)

Finalized for submission on 31 Dec 2025

Table of Contents

Abbreviations and definitions

Abbrev	Longform (= translation)
+x	adding the eXecutable permission
-d (in docker)	--detach
-d (in timescaledb)	database name
-e	exists
-f	--file
-i	ignore case
-L	Local Port Forwarding
-R	Recursive
-T	--no-TTY (= disable pseudo-TTY allocation)
-U	Username
-v	invert match
.htpasswd	HyperText password
.md	markdown
.sh	shell script
BSc	Bachelor of Science
cd	change directory
chmod	change mode
chown	change owner
cp	copy
csv	comma-separated values
DLBDSEDE02	Distance Learning Bachelor Data Science Elective Data Engineering – course #2
DLQ	Dead Letter Queue
docs	documents
DR/dr(.)	doctor
env(var)	environment(variable)
exec	execute
grep	global regular expression print
id	identifier
infra	infrastructure
init	initialization
JDBC	Java Database Connectivity
jq	JSON query
JSON/json	JavaScript Object Notation
ln -s	symlink, symbolic link
N	some number
ps	process status
psql	PostgreSQL (client program)
scp	secure copy
SQL	Structured Query Language
SSH	Secure Shell
sudo	superuser do
tmp	temporary
ts	timestamp
ui	user interface
VM	virtual machine
yml	Yet Another Markup Language

VLC – Technical description on how to use the code

In the following, we are providing a brief description on how the pipeline is envisioned to function.

For details, please check the main README.md in the project root and follow the instructions to reproduce the setup. It might also be interesting to have a look at the subREADMEs provided in "docs\README.md" focusing on the project as a whole, "producer\README.md" providing information on field renaming among other topics, "schemas\README.md" explaining the role of Apache Avro within the repository and "tests\README.md" with data on how tests were set up to keep development entropy in check, while it might also be a good idea to open "db\direct_sql_queries.txt" where we stashed example queries written during the creation of the dashboards that you can try out directly in timescaledb.

1. Roles of components

1. Local dev station: runs SSH client and optional port-forward (`ssh -L...`)

2. VM: always-on Linux box in the sky that runs the pipeline 24/7

3. Docker daemon: long-running supervisor starting containers

4. SSH: a network protocol that establishes encrypted connections between computers for secure remote access

5. Python producers: incontainer processes protected by resilience wrapper (backoff, DLQ, inflight limiting).

2. One-time bring-up without ui beast of burden

2.0 Provision virtual setup

az group create --name vlc-rg --location spaincentral

az deployment group create --resource-group vlc-rg --template-file infra/main.bicep --parameters adminUsername=<username> sshPublicKey="$(cat <file_location>)"

2.1 Ensure repo & envvars are in place

cd vlc

git pull

scp .env

[-e compose/.env] || ln -s ../.env compose/.env

sudo chown -R someuser:somegroup connect/secrets

chmod +x scripts/sync-dev-secrets.sh

./scripts/sync-dev-secrets.sh

chmod 644 compose/.htpasswd

docker compose -f compose/docker-compose.yml down

docker compose -f compose/docker-compose.yml --profile infra --profile schema --profile producer up -d --build

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -f /docker-entrypoint-initdb.d/001-extensions.sql

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -f /docker-entrypoint-initdb.d/010-bootstrap.sql

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -f /docker-entrypoint-initdb.d/020-views.sql

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -f /docker-entrypoint-initdb.d/020-vlc-password.override.sql

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -f /docker-entrypoint-initdb.d/099-final-checks.sql

2.2 Backfill run

docker cp backfill/backfill.sql $(docker compose -f compose/docker-compose.yml ps -q timescaledb):/tmp/

docker cp backfill/hourly_2016_2020.csv $(docker compose -f compose/docker-compose.yml ps -q timescaledb):/tmp/

docker compose -f compose/docker-compose.yml cp backfill/hourly_2021_2022.csv timescaledb:/tmp/hourly_2021_2022.csv

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -f /tmp/backfill.sql

docker compose -f compose/docker-compose.yml exec timescaledb rm /tmp/backfill.sql /tmp/hourly_2016_2020.csv /tmp/hourly_2021_2022.csv

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -f /docker-entrypoint-initdb.d/030-consolidate-station-ids.sql

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT fiwareid, COUNT(*) as records, MIN(ts)::date as first_seen, MAX(ts)::date as last_seen FROM air.hyper GROUP BY fiwareid ORDER BY fiwareid;"

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT fiwareid, COUNT(*) as records, MIN(ts)::date as first_seen, MAX(ts)::date as last_seen FROM weather.hyper GROUP BY fiwareid ORDER BY fiwareid;"

2.3 Fire up kafka

chmod +x scripts/bootstrap_kafka.sh

BROKER_CONTAINER=kafka BOOTSTRAP=kafka:9092 ./scripts/bootstrap_kafka.sh 2>&1 | grep -v "WARNING.*metric names"

The script waits for Kafka to be healthy, creates `vlc.air`, `vlc.weather` and Connect internal topics, then upserts the JDBC sink connectors using `connect/config/jdbc-sink.timescale.air.json` and `...weather.json`.

3. Check operations

cd vlc

git status

docker compose -f compose/docker-compose.yml ps

docker compose -f compose/docker-compose.yml logs kafka --tail=80

docker inspect --format '{{json .State.Health}}' $(docker compose -f compose/docker-compose.yml ps -q kafka) | jq

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT count(*) FROM air.hyper;"

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT count(*) FROM weather.hyper;"

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT max(ts) FROM air.hyper;"

docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT max(ts) FROM weather.hyper;"

docker compose -f compose/docker-compose.yml --profile infra --profile producer logs -t air-producer --tail=50

docker compose -f compose/docker-compose.yml --profile infra --profile producer logs -t weather-producer --tail=50

At this point, you should see logs like `produced N; offset=...; seen=...`, `no new records` between polls, + DLQ stats if something failed.

```
docker compose -f /opt/vlc/compose/docker-compose.yml --profile infra --profile producer ps -a 2>&1 | grep -iE 'sink|connect|NAME'
```

```
docker compose -f /opt/vlc/compose/docker-compose.yml --profile infra --profile producer logs air-sink --tail=30
```